Basi di dati — Tecnologie

Università di Verona
Imbriani Paolo - VR500437
Professor Sara Migliorini

March 10, 2026

Contents

1 SQL **3**

1.1 Istruzioni . . . 3
1.2 SELECT . . . 4
1.3 FROM . . . 4
1.4 WHERE . . . 5
1.4.1 LIKE . . . 5
1.4.2 BETWEEN . . . 5
1.4.3 SIMILAR TO . . . 5
1.4.4 IN . . . 6
1.4.5 IS NULL . . . 6
1.5 SQL e Algebra Relazionale . . . 6
1.5.1 Duplicati . . . 6
1.5.2 Selezione, Proiezione, Join . . . 7
1.6 JOIN . . . 7
1.6.1 INNER JOIN . . . 7
1.6.2 LEFT [OUTER] JOIN . . . 8
1.6.3 RIGHT [OUTER] JOIN . . . 8
1.6.4 FULL [OUTER] JOIN . . . 8
1.7 ALIAS . . . 8
1.8 ORDER BY . . . 8
1.9 OPERAZIONI INSIEMISTICHE . . . 9
1.10 OPERATORI AGGREGATI . . . 9
1.10.1 COUNT . . . 9
1.10.2 MAX, MIN, AVG, SUM . . . 9
1.11 OPERATORI DI RAGGRUPPAMENTO . . . 10
1.11.1 GROUP BY . . . 10
1.11.2 HAVING . . . 10
1.12 Interrogazioni nidificate . . . 11
1.12.1 Nidificazione nella clausola WHERE . . . 11
1.12.2 Nidificazione nidifiicate Binding e EXISTS . . . 11
1.13 VIEW . . . 13
1.14 Esercizi . . . 13

1 SQL

SQL è il linguaggio di riferimento per le basi di dati relazionali. Originariamente SQL era l'acronimo di "Structured Query Language", ma oggi è semplicemente SQL. SQL ha varie funzionalità:

- **Data Definition Language (DDL)**: consente di definire la struttura del database, come la creazione di tabelle, l'aggiunta di colonne e la definizione di chiavi primarie.
- **Data Manipulation Language (DML)**: consente di manipolare i dati all'interno del database, come l'inserimento, l'aggiornamento e la cancellazione di record.
- **Data Query Language (DQL)**: consente di interrogare il database per recuperare informazioni specifiche.

Ne esistono inoltre varie versioni. In laboratorio useremo SQL-2 poiché sappiamo che è supportato da tutti i DBMS.

1.1 Istruzioni

La sintassi di base è:

```
SELECT [ DISTINCT ]
[ * | expression [[ AS] output_name ] [, ...] ]
[ FROM from_item [, ...] ]
[ WHERE condition ]
[ GROUP BY grouping_element [, ...] ]
[ HAVING condition [, ...] ]
[ { UNION | INTERSECT | EXCEPT } [ DISTINCT ] other_select ]
[ ORDER BY expression [ ASC | DESC | USING operator ]]
```

- expression è un' espressione che determina un attributo.
- from_item è un' espressione che determina una sorgente per gli attributi.
- condition è un' espressione booleana per selezionare i valori degli attributi.
- grouping_element è un'espressione per poter eseguire operazioni su più valori di un attributo e considerare il risultato
- SELECT, FROM e WHERE sono chiamate clausole.
- Estrae dal prodotto cartesiano delle tabelle elencate nella clausola FROM, le righe che soddisfano la condizione specificata nella clausola WHERE.

– Il risultato dell'interrogazione è una tabella con una riga per ogni riga prodotta dalla clausola FROM ed eventualmente filtrata dalla clausola WHERE (se esiste), le cui colonne dipendono dalla lista degli attributi nella clausola SELECT.

1.2 SELECT

```
SELECT [ DISTINCT ]
[{* | expression [[ AS] output_name ]} [, ...]]
```

- è un'abbreviazione per indicare tutti gli attributi delle tabelle.
- expression è un'espressione che coinvolge gli attributi della tabella (può anche essere semplicemente il nome di un attributo).
- output_name è il nome assegnato all'attributo che conterrà il risultato della valutazione dell'espressione expression nella relazione risultato.
- DISTINCT: se presente richiede l'eliminazione delle tuple duplicate.

1.3 FROM

```
FROM from_item [, ...]
```

1. `table_name [[AS] alias [(column_alias [, ...])]`
   - Se sono presenti due o più tabelle, si esegue il prodotto cartesiano tra tutte le tabelle. Se ci sono attributi con lo stesso nome su tabelle diverse, nella SELECT questi attributi sono identificati anteponendo il nome della tabella e un ".": nomeTabella.nomeAttributo
2. `(other_select) [AS] nomeRisultato [(column_alias [,...])]`
   - è una SELECT innestata. Il risultato della SELECT interna è lo schema con nome nomeRisultato su cui fare la SELECT corrente.
3. `from_item [NATURAL] join_type from_item [ON condition]`
   - è la clausola JOIN.
   - Metodo più sofisticato che permette di selezionare un sottoinsieme del prodotto cartesiano di due o più tabelle. join_type specifica il tipo di join. I tipi principali sono riassunti nelle slide seguenti.

1.4 WHERE

```
WHERE condition
```

La clausola WHERE ammette come argomento un'espressione booleana costruita combinando predicati semplici con gli operatori AND, OR e NOT

- AND: seleziona le righe in cui tutti i predicati sono veri
- OR: seleziona le righe in cui almeno un predicato è vero
- NOT: seleziona le righe in cui il predicato è falso

Ciascun predicato semplice usa gli operatori =, <>, <, >, <= e >= p er confrontare un'espressione costruita a partire dal valore di un attributo con un valore costante, oppure un'altra espressione. Per esprimere la precedenza degli operatori AND e OR si fa uso della parentesi.

1.4.1 LIKE

Per il confronto tra stringhe, SQL mette a disposizione anche l'operatore LIKE che consente verificare se una stringa rispetta un certo "pattern" (oppure ILIKE per case insentive).
Due caratteri speciali per il confronto "_" e "%":

- "_": confronto con un carattere arbitrario
- "%": confronto con una stringa di lunghezza arbitraria (anche 0) di caratteri arbitrari

'_o%i' = stringa che ha una o in seconda posizione, seguita da una sequenza qualsiasi di caratteri (anche vuota) e che termina con una i.

1.4.2 BETWEEN

L'operatore BETWEEN consente di verificare se un valore è compreso tra due valori

```
WHERE attributo [NOT] BETWEEN valore1 AND valore2;
```

Il predicato ritorna true se il valore di attributo è compreso tra valore1 e valore2 (inclusi). Se è presente NOT, ritorna true se il valore di attributo è minore di valore1 o maggiore di valore2.

1.4.3 SIMILAR TO

L'operatore SIMILAR TO è un LIKE più espressivo che accetta, come pattern, un sottoinsieme delle espressioni regolari POSIX (versione SQL).
Esempi di componenti di espressioni regolari:

- _: confronto con un carattere arbitrario
- %: 0 o più caratteri arbitrari

- : ripetizione dell'elemento precedente (carattere o gruppo) 0 o più volte
- +: ripetizione dell'elemento precedente (carattere o gruppo) 1 o più volte
- $\{n, m\}$: ripetizione del precedente match almeno n e non più di m volte.
- [...]: elenco di caratteri ammissibili

1.4.4 IN

Nella clausola WHERE può apparire l'operatore [NOT] IN per testare l'appartenenza di un valore ad un insieme.

```
WHERE attributo [NOT] IN ( valore [, ...] );
```

1.4.5 IS NULL

Per verificare se un attributo ha un valore nullo o meno si utilizza il predicato IS NULL.

- Attributo IS NULL: ritorna true se l'attributo ha valore nullo
- IS NOT NULL: è la negazione, ritorna true se l'attributo ha valore diverso da nullo

```
WHERE attributo IS [ NOT ] NULL;
```

SQL-2 implementa la logica a 3 valori:

- Un predicato semplice restituisce il valore NULL quando uno qualsiasi dei termini del predicato ha valore nullo.
- Il predicato IS NULL è una eccezione, restituendo sempre il valore vero oppure false, mai il valore NULL.

1.5 SQL e Algebra Relazionale

1.5.1 Duplicati

Una significativa differenza tra SQL e algebra relazionale è la gestione dei duplicati. L'operazioni di eliminazione delle righe doppie (ad esempio al seguito di una selezione con where) è molto costosa, quindi di default SQL mantiene le righe duplicate.

- Il costrutto DISTINCT dopo la parola chiave SELECT permette di eliminare i duplicati.
- Il costrutto ALL permette invece di mantenerli, ma questa è già la scelta di default.

1.5.2 Selezione, Proiezione, Join

- L'istruzione SELECT, con una sola relazione nella clausola FROM permette di realizzare: selezioni, proiezioni, ridenominazioni.
- con più relazioni nella clausola FROM si realizzano join (e prodotti cartesiani)

In SQL il join si può definire in due modi diversi:

- Il primo non richiede l'uso di nuova sintassi, e specifica il legame tra le tabelle coinvolte a livello della clausola WHERE.
- Il secondo usa una sintassi riservata a livello di clausola FROM (useremo questo).

La clausola FROM ammette come argomento: `table_name [[AS] name]] [, ...]`. Si è visto che se sono presenti due o più nomi di tabelle, si esegue il prodotto cartesiano tra tutte le tabelle e lo schema del risultato può contenere tutti gli attributi del prodotto cartesiano. Il prodotto cartesiano di due o più tabelle è un **CROSS JOIN**. A partire da SQL-2, esistono altri tipi di JOIN (join_type): INNER JOIN, LEFT [OUTER] JOIN, RIGHT [OUTER] JOIN e FULL [OUTER] JOIN.

1.6 JOIN

```
table_name [ NATURAL ] join_type table_name [ON join_condition [, ...]].
```

dove join_condition è un'espressione booleana che seleziona le tuple del join da aggiungere al risultato. Le tuple selezionate possono essere poi filtrate con la condizione della clausola WHERE. Il join si può spostare nella clausola from, a livello delle tabelle (nello svolgere gli esercizi, useremo questa sintassi dedicata).

- ```
  SELECT ListaAttributi
  FROM Tabella1 JOIN Tabella2 ON ( CondizioneDiJoin )
  [ WHERE AltraCondizione]
  ```

- ```
  SELECT i.Nome, d.Città
  FROM Impiegato AS i JOIN Dipartimento as d ON ( i.Dipart = d.Nome )
  ```

Le parole OUTER, LEFT, RIGHT e FULL sono opzionali. Se non sono presenti, si intende INNER JOIN.

1.6.1 INNER JOIN

```
table1 [ INNER ] JOIN table2 ON join_condition [, ...].
```

Rappresenta il tradizionale θ-join dell'algebra relazionale. Combina ciascuna riga r1 di table1 con ciascuna riga di table2 che soddisfa la condizione della clausola ON.

1.6.2 LEFT [OUTER] JOIN

```
table1 LEFT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga r1 di table1 che non soddisfa la condizione con qualsiasi riga di table2, si aggiunge una riga al risultato con i valori di r1 e assegnando NULL agli altri attributi...

> **Warning 1.1** (Attenzione)
> Il LEFT [OUTER] JOIN non è simmetrico! Con le medisime tabelle si possono ottenere risultati diversi invertendo l'ordine delle tabelle nel join!

1.6.3 RIGHT [OUTER] JOIN

```
table1 RIGHT [ OUTER ] JOIN table2 ON join_condition [, ...].
```

Si esegue un INNER JOIN. Poi, per ciascuna riga r2 di table2 che non soddisfa la condizione con qualsiasi riga di table1, si aggiunge una riga al risultato con i valori di r2 e assegnando NULL agli altri attributi..

1.6.4 FULL [OUTER] JOIN

```
table1 FULL [ OUTER ] JOIN table2 ON join_condition [, ...].
```

È equivalente a: INNER JOIN + LEFT OUTER JOIN + RIGHT OUTER JOIN.

1.7 ALIAS

SQL permette di associare un nome alternativo (alias) alle tabelle che compaiono nella clausola FROM. Il nome viene usato per far riferimento alla tabella nel contesto dell'interrogazione. Questo perché:

- Far riferimento alla tabella in modo compatto.
- Fare accesso più volte alla stessa tabella

1.8 ORDER BY

Come abbiamo visto, una relazione è un insieme di tuple non ordinato. Nella realtà, però, sorge spesso il bisogno di costruire un ordine sulle righe. SQL permette di specificare un ordinamento sulle righe dell'output grazie alla clausola ORDER BY.

```
ORDER BY attributo [{ ASC | DESC }] [, ... ];
```

Le righe vengono ordinate per il primo attributo nell'elenco, a parità di valore del primo attributo si usa il secondo, e così via in sequenza.

1.9 OPERAZIONI INSIEMISTICHE

SQL mette a disposizione degli operatori insiemistici.

```
SELECT
{ UNION | INTERSECT| EXCEPT
[ALL] SELECT }
```

Gli operatori insiemistici, al contrario del resto del linguaggio, assumono come default di eseguire un'eliminazione dei duplicati, a meno che si usi ALL.
Sono sufficienti domini compatibili ed ugual numero di attributi

- Ordine posizionale: la corrispondenza degli attributi si basa sulla posizione
- Se i nomi degli attributi sono diversi di solito il sistema usa il nome del primo operando

1.10 OPERATORI AGGREGATI

Una delle più importanti estensioni di SQL rispetto all'algebra relazionale: **COUNT, SUM, AVG, MIN, MAX**. In algebra relazionale tutte le condizioni specificate vengono valutate su una tupla alla volta. Gli operatori di aggregazione permettono di valutare proprietà che dipendono da insiemi di tuple. Prima viene eseguita l'interrogazione considerando solo le clausole FROM e WHERE, quindi l'operatore di aggregazione viene applicato alla tabella contenente il risultato.

1.10.1 COUNT

L'operatore COUNT conta il numero di righe di una tabella o il numero di valori di uno o più attributi:

```
COUNT ( <* | [DISTNCT | ALL] ListaAttributi> )
```

- *: conta il numero di righe della tabella
- DISTINCT ListaAttributi: conta il numero di valori diversi degli attributi ListaAttributi
- ALL ListaAttributi: conta il numero di righe che hanno valori diversi da NULL negli attributi in ListaAttributi (comportamento di default)

1.10.2 MAX, MIN, AVG, SUM

```
<SUM | MAX | MIN | AVG > ( [DISTINCT | ALL] AttrExpr )
```

- SUM: calcola la somma dei valori di AttrExpr
- MAX: calcola il valore massimo di AttrExpr

- MIN: calcola il valore minimo di AttrExpr
- AVG: calcola la media dei valori di AttrExpr

SUM e **AVG** ammettono come argomento solo espressioni che rappresentano valori numerici o intervalli di tempo. **MAX** e **MIN** ammettono come argomento solo espressioni su cui è definito un ordinamento (quindi anche stringhe). **DISTINCT** elimina i duplicati mentre **ALL** trascura solo i valori NULL. (DISTINCT e ALL non hanno effetto su MIN e MAX)

```
SELECT Cognone, Nome, MAX( I.Stipendio )
FROM Impiegato as I JOIN Diparmento as D
ON I.Dipart = D.Nome
WHERE D.Città = 'Milano'
```

Warning 1.2

la precedente query non è corretta! Gli operatori aggregati non sono un meccanismo di selezione, ma solo funzioni che restituiscono un valore quando sono applicate ad un insieme. Cognome e Nome restituiscono un valore per ogni tupla selezionata, mentre MAX() restituisce un valore unico per tutto l'insieme. Per risolvere questa situazione serve il meccanismo di raggruppamento che non vediamo.

1.11 OPERATORI DI RAGGRUPPAMENTO

1.11.1 GROUP BY

Gli operatori aggregate possono essere applicati separatamente a sottoinsiemi di righe. La clausola GROUP BY permette di specificare come dividere le tabelle in sottoinsiemi. La clausola permette di specificare un insieme di attributi e l'interrogazione raggrupperà le righe che possiedono gli stessi valori per questo insieme di attributi. In una interrogazione che fa uso della clausola GROUP BY può comparire come argomento della SELECT solo un sottoinsieme degli attributi usati nella clausola GROUP BY oppure operatori di aggregazione.

1.11.2 HAVING

È possibile applicare delle selezioni ai gruppi generati tramite la clausola GROUP BY. La clausola HAVING permette di specificare delle condizioni che devono essere soddisfatte dai gruppi generati dopo l'esecuzione del raggruppamento tramite GROUP BY.

Osservazione 1.1

- La clausola WHERE applica delle selezioni su SINGOLE RIGHE, prima dell'eventuale raggruppamento tramite GROUP BY.

- La clausola HAVING applica delle selezioni su GRUPPI, generati a seguito dell'esecuzione del raggruppamento tramite GROUP BY.

1. WHERE → seleziona le righe che verranno raggruppate
2. GROUP BY → raggruppa le righe selezionate
3. HAVING → seleziona i gruppi che fanno parte del risultato

1.12 Interrogazioni nidificate

Una interrogazione nidificata è una interrogazione che contiene al suo interno altre interrogazioni. La nidificazione può avvenire sia nella clausola SELECT, che nella clausola FROM, che nella clausola WHERE. Il caso tipico è la nidificazione nella clausola WHERE, dove un predicato contiene un confronto tra un valore (espressione su una singola riga) ed il risultato di un'interrogazione SQL.

1.12.1 Nidificazione nella clausola WHERE

Se in un predicato si confronta un attributo o una espressione che produce un valore singolo con il risultato di un'interrogazione, sorge il problema di disomogeneità dei termini del confronto. I normali operatori di confronto ($=, <>, <, >, <=, =>$) possono essere estesi con le parole chiave ANY e ALL.

- ANY: la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo (o espressione) e **ALMENO** uno degli elementi restituiti dall'interrogazione.
- ALL: la riga soddisfa la condizione se risulta vero il confronto (con l'operatore specificato) tra il valore dell'attributo (o espression) e **TUTTI** gli elementi restituiti dall'interrogazione

Per le interrogazioni nidificate viste fino ad ora, l'interrogazione può essere eseguita una volta sola ed il risultato ottenuto può essere salvato in una tabella temporanea usata per eseguire ciascun confronto con le righe dalla tabella esterna. Talvolta però l'interrogazione nidificata fa riferimento al contesto dell'interrogazione che la racchiude attraverso l'uso di una variabile (*passaggio di binding*). L'interrogazione nidificata dovrà essere valutata separatamente per ciascuna riga prodotta dalla valutazione della query esterna.

1.12.2 Nidificazione nidifiicate Binding e EXISTS

L'operatore logico **EXISTS** riceve come parametro un'interrogazione nidificata e restituisce il valore vero solo se l'interrogazione fornisce un risultato non vuoto. Questo operatore può essere usato in modo significativo solo quando si ha un passaggio di binding tra la query esterna e quella interna.

```
SELECT * FROM Persona P
WHERE EXISTS (SELECT * FROM Persona P1 WHERE P1.Nome
    = P.Nome AND P1.Cognome = P.Cognome AND P1.CodFiscale
    <> P.CodFiscale );
```

Nel codice sopra si estraggono le persone che hanno degli omonimi (ovvero persone con lo stesso nome e cognome, ma codice fiscale diverso). In questo prossimo codice si vanno ad estrarre le persone che non hanno degli omonimi (ovvero persone con lo stesso nome e cognome, ma codice fiscale diverso)

```
SELECT * FROM Persona P
WHERE NOT EXISTS (SELECT * FROM Persona P1
    WHERE P1.Nome = P.Nome AND
    P1.Cognome = P.Cognome AND
    P1.CodFiscale <> P.CodFiscale );
```

Le interrogazioni nidificate possono essere specificate anche come elemento della clausola SELECT. Spesso l'uso di query nidificate nella SELECT sostituiscono l'uso del JOIN. Per esempio, estrarre per ogni cantante il numero di canzoni di cui è autore.

```
SELECT DISTINCT Nome, NumCanzoni =
    (SELECT COUNT(*) FROM Autore A
    WHERE A.Nome = C.Nome)
    FROM Cantante C
```

oppure

```
SELECT DISTINCT C.Nome, COUNT(*) as NumCanzoni
FROM Cantante C JOIN Autore A
ON C.Nome = A.Nome
GROUP BY C.Nome
```

Osservazione 1.2

Ricordiamo che è necessario che l'interrogazione nidificate restituisca una sola tupla (riferimenti tramite variabile alla interrogazione esterna).

Le interrogazioni nidificate possono essere utilizzate nella clausola FROM come sorgente dati su cui eseguire l'interrogazione. È possibile creare una catena di interrogazioni, dove ciascuna interrogazione utilizza il risultato della precedente come una tabella di base. Per esempio: si estrae per ogni cantante che è anche autore di canzoni il numero di canzoni di cui è rispettivamente interprete e autore.

```
SELECT DISTINCT C.Nome, C.NumCantate, A.NumScritte
FROM
(SELECT Nome, Count(*) as NumCantate FROM Cantante) C
JOIN
(SELECT Nome, Count(*) as NumScritte FROM Autore ) A
ON C.Nome = A.Nome
```

Prima vengono eseguite le interrogazioni nella clausola FROM e i risultati di queste vengono usati come normali tabelle.

1.13 VIEW

Le viste permettono di salvare una query che genera una tabella temporanea.

```sql
CREATE VIEW <nome_vista> AS (<QUERY>)
```

Dato un nome alla query, la possiamo usare come se fosse il nome di una tabella.

1.14 Esercizi

Esempio 1.1

We have here three tables:

- **TURISTA** con gli attributi USERNAME, COGNOME, NOME, DATA_NASCITA, NAZIONALITA
- **ATTRAZIONE** con gli attributi CODICE, NOME, DESCRIZIONE, LATITUDINE, LONGITUDINE, INDIRIZZO, CITTA, PROV
- **RECENSIONE** con gli attributi ATTRAZIONE, TURISTA, TIMESTAMP, TITOLO, TESTO, VOTO

1. Trovare tutte le attrazioni della città di Verona il cui nome inizia con la lettera *f* maiuscola o minuscola.

```sql
SELECT *
FROM ATTRAZIONE
WHERE CITTA = 'Verona' AND NOME ILIKE 'f%';
```

2. Trovare tutti i turisti che nel 2025 non hanno mai rilasciato recensioni con un voto > 2 per attrazioni fuori dalla provincia di Verona.

```sql
SELECT T.USERNAME FROM TURIST T
EXCEPT (
    SELECT R.TURISTA
    FROM RECENSIONE R JOIN ATTRAZIONE A ON R.ATTRAZIONE = A.CODICE
```

```sql
    WHERE
        R.VOTO > 2 AND
        A.PROVINCIA <> 'VERONA' AND
        EXTRACT (YEAR FROM R.TIMESTAMP) = '2025'
)
```

Osservazione 1.3

Le due query devono essere compatibili.

Questa query nidificata che restituisce i turisti che hanno rilasciato recensioni con voto > 2 per attrazioni fuori dalla provincia di Verona nel 2025 e la chiararemo Q1. Oppure

```sql
SELECT T.USERNAME, T.NOME, T.COGNOME FROM TURIST T
NOT IN (
    SELECT R.TURISTA
    FROM RECENSIONE R JOIN ATTRAZIONE A ON R.ATTRAZIONE = A.CODICE
    WHERE
        R.VOTO > 2 AND
        A.PROVINCIA <> 'VERONA' AND
        EXTRACT (YEAR FROM R.TIMESTAMP) = '2025'
)
```

Osservazione 1.4 (Q)

esta query non richiede che gli schemi siano compatibili,e permette di aggiungere facilmente altri attributi alla query esterna

3. Trovare tutte le attrazioni che hanno ricevuto nel 2025 un numero di recensioni superiore a quelle ricevute nel 2024.

   - Date tutte le righe di R1
   - Le divide per attrazioni in un gruppo per ogni ATTRAZIONE
   - Applica count su ogni gruppo

```sql
SELECT COUNT(*) AS NumRecensioni, R1.ATTRAZIONE
FROM RECENSIONE R1
WHERE EXTRACT (YEAR FROM R1.TIMESTAMP) = '2025'
GROUP BY R1.ATTRAZIONE
HAVING
    NumRecensioni > (
        SELECT COUNT(*) AS NumRecensioni2024
        FROM RECENSIONE R2
        WHERE
```

```
            EXTRACT (YEAR FROM R2.TIMESTAMP) = '2024' AND
            R1.ATTRAZIONE = R2.ATTRAZIONE
    );
```

Nella query annidata non ci sta bisogno di fare il `GROUP BY` poiché abbiamo già fatto il DATA BINDING tra R1 e R2 con `R1.ATTRAZIONE = R2.ATTRAZIONE` e di conseguenza R2.ATTRAZIONE è già un valore singolo e non un insieme di valori.

4. Trovare la media dei voti massimi rilasciati dagli utenti.

   - Calcolare il voto massimo rilasciato da ciascun utente $\{max_1, \ldots, max_n\}$
   - Calcolare la media dei valori in $\{max_1, \ldots, max_n\}$

   Warning 1.3

   NO! Questa query non si può fare

```
SELECT AVG(MAX(VOTO))
...
```

   Procediamo in questa maniera:

```
SELECT TURISTA, MAX(VOTO) AS MaxVoto
FROM RECENSIONE
GROUP BY TURISTA
```

   e poi facciamo la media:

```
SELECT AVG(T.MaxVoto)
FROM (
    -- Prendiamo il voto massimo rilasciato da ciascun utente
    SELECT TURISTA, MAX(VOTO) AS MaxVoto
    FROM RECENSIONE R
    GROUP BY R.TURISTA
) AS T
```

Esempio 1.2

Consideriamo le seguenti tre tabelle:

- AUTO(TARGA, NUM_TELAIO, MODELLO, COLORE, KM, CILINDRATO, POSTI, PROPRIETARIO)
- PARCHEGGIO(NUMERO, COMUNE, INDIRIZZO, COPERTO)
- PRENOTAZIONE(CODICE, PARCHEGGIO, POSTO, AUTO,

DATA_ORA_INIZIO, DATA_ORA_FINE, TIPO, PAGATO)

Dove

$$\text{PRENOTAZIONE.PARCHEGGIO} \rightarrow \text{PARCHEGGIO}$$

$$\text{PRENOTAZIONE.AUTO} \rightarrow \text{AUTO}$$

Osservazione 1.5

Ricordiamo che SQL è case insentive.

1. Trovare tutti i parcheggi coperti di Verona

```sql
SELECT *
FROM PARCHEGGIO
WHERE COMUNE = 'VERONA' AND COPERTO;
```

2. Trovare tutti i parcheggi in cui non sono mai sstate fatte prenotazioni online nel 2024. Per ciascun parcheggio riportarare il nome ed il numero di prenotazioni complessive.

```sql
SELECT P.NOME, COUNT(*) AS NumeroPrenotazioni
FROM PARCHEGGIO P JOIN PRENOTAZIONE R ON (P.CODICE = R.PARCHEGGIO)
WHERE P.CODICE NOT IN (
    -- Query annidata che trova tutti i parcheggi che hanno avuto
    prenotazioni online nel 2024
    SELECT R1.PARCHEGGIO
    FROM PRENOTAZIONI R1
    WHERE
        EXTRACT (YEAR FROM DATA_ORA_INIZIO) = '2024' AND
        TIPO = 'ONLINE'
)
GROUP BY P.NOME;
```

3. Trovare tutti i parcheggi in cui vengono effettuate più prenotazioni online che prenotazioni fisiche.

```sql
SELECT PR.PARCHEGGIO
FROM PRENOTAZIONE PR
WHERE R.TIPO = 'ONLINE'
GROUP BY R.PARCHEGGIO
HAVING COUNT(*) > (
    SELECT COUNT(*)
    FROM PRENOTAZIONE R2
    WHERE
        R2.TIPO = 'FISICA' AND
        R2.PARCHEGGIO = R.PARCHEGGIO -- DATA BINDING
)
```

Osservazione 1.6

La query interna è eseguita per ogni riga della query esterna.

4. Trovare quanti parcheggi del comune di Verona hanno avuto un numoro medio di prenotazioni nel 2024 superiore al numero medio di prenotazioni complessive del 2024 dei parcheggi del comune di Padova.

   (a) Calcolare # prenotazioni nel 2025 per parcheggio per ogni parcheggio di Verona

   (b) Calcolare # prenotazioni nel 2025 per parcheggio per ogni parcheggio di Padova

   (c) Calcolare la media su (b)

```sql
CREATE VIEW NPRENOTAZIONI AS
(
    SELECT R.PARCHEGGIO, P.COMUNE, COUNT(*) AS NUM_PR
    FROM PARCHEGGIO P JOIN PRENOTAZIONE R ON (P.CODICE = R.PARCHEGGIO)
    WHERE EXTRACT (YEAR FROM R.DATA) = '2024'
    GROUP BY R.PARCHEGGIO, P.COMUNE
)

SELECT N.PARCHEGGIO
FROM NPRENOTAZIONI AS N
WHERE
    N.NUM_PR > (
        -- numero medio di prenotazioni dei parcheggi di PD nel 2024
        -- (ritorna un solo valore)
        SELECT AVG(N1.NUM_PR)
        FROM NPRENOTAZIONI AS N1
        WHERE N1.COMUNE = 'PADOVA'
    ) AND
    N.COMUNE = 'VERONA'; -- Per ogni parcheggio VR guardo quali hanno #
     prenotazioni maggiori
```